

Running Notes- LangChain & Agentic AI – From Basics to Advanced 23

RunnableSequence & Sequential Workflows

1. Quick Day 1 Revision

On Day 1, we learned:

chain = prompt | model | parser

Flow:

Prompt



Model



Parser

Main Rule

Output of Left



Input of Right

Simple logic:

prompt | model | parser

prompt output



model input

model output



parser input

2. Quick Day 2 Revision

We learned that our chain follows the **Runnable interface**.

Therefore, we can use:

`chain.invoke(...)`

for **one input**.

`chain.batch(...)`

for **multiple inputs**.

`chain.stream(...)`

for **progressive output**, when streaming is supported.

Today, we concentrate on:

How are multiple Runnables executed one after another?

Answer:

RunnableSequence

3. What Is Sequential Execution?

Sequential execution means performing operations one after another in a defined order.

Real-World Example — Preparation of Tea

Boil Water



Add Tea



Add Milk



Filter



Serve

Every step happens after the previous step.

Similarly:

chain = prompt | model | parser

executes components sequentially.

Important Note

In LangChain, we can implement sequential workflows by using:
RunnableSequence

4. What Is RunnableSequence?

RunnableSequence is a sequence of Runnables where the output of one Runnable becomes the input of the next Runnable.

Basic Flow

INPUT



Runnable 1



Runnable 2



Runnable 3



OUTPUT

For our normal LangChain application:

Dictionary



PromptTemplate



Chat Model



StrOutputParser



String

5. Our Existing Chain Is Already a RunnableSequence

Consider:

```
chain = prompt | model | parser
```

This itself creates a sequential pipeline.

We can check its type:

```
print(type(chain))
```

6. Program — Check RunnableSequence Type

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
prompt = PromptTemplate.from_template(  
    "Explain {topic} in simple English."  
)
```

```
model = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
parser = StrOutputParser()
```

```
chain = prompt | model | parser
```

```
print(type(chain))
```

The complete pipeline is represented as a Runnable sequence.

7. Is RunnableSequence Created Only with | ?

No.

We can also create it explicitly.

Import:

```
from langchain_core.runnables import RunnableSequence
```

Then:

```
chain = RunnableSequence(  
    prompt,  
    model,  
    parser  
)
```

8. Explicit RunnableSequence Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableSequence
```

```
prompt = PromptTemplate.from_template(
    "Explain {topic} in very simple English."
)
```

```
model = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
```

```
parser = StrOutputParser()
```

```
chain = RunnableSequence(
    prompt,
```

```
    model,  
    parser  
)
```

```
result = chain.invoke(  
    {"topic": "Python"}  
)
```

```
print(result)
```

9. RunnableSequence() vs |

Both can create a sequential Runnable pipeline.

Direct Constructor Approach

```
chain = RunnableSequence(  
    prompt,  
    model,  
    parser  
)
```

LCEL Pipe Operator Approach

chain = prompt | model | parser

Important Note

RunnableSequence(...)

is the direct constructor approach.

The | operator is the **LCEL composition style** used to build the sequence.

The | style is commonly preferred because it clearly looks like a pipeline.

prompt



model



parser

10. Lambda Function

A lambda function is also called:

- Lambda Function
- Anonymous Function
- Nameless Function

Example:

```
m1 = lambda x: 2 * x
```

```
print(m1(2))
```

Output:

```
4
```

11. Converting Normal Python Functions into Runnables

LangChain provides:

RunnableLambda

RunnableLambda converts a Python callable into a Runnable.

Import:

```
from langchain_core.runnables import RunnableLambda
```

Example:

```
add_ten = RunnableLambda(  
    lambda x: x + 10  
)
```

Now add_ten becomes a Runnable.

12. Simple RunnableLambda Program

```
from langchain_core.runnables import RunnableLambda
```

```
add_ten = RunnableLambda(  
    lambda x: x + 10  
)
```

```
print(type(add_ten))
```

```
multiply_two = RunnableLambda(  
    lambda x: x * 2  
)
```

```
chain = add_ten | multiply_two
```

```
result = chain.invoke(5)
```

```
print(result)
```

Execution

Input:

5

Step 1:

$5 + 10$

↓

15

Step 2:

15×2

↓

30

Final Output:

30

13. Cleanest Example to Understand RunnableSequence

```
from langchain_core.runnables import RunnableLambda
```

```
add_ten = RunnableLambda(  
    lambda x: x + 10  
)
```

```
multiply_two = RunnableLambda(  
    lambda x: x * 2  
)
```

```
subtract_five = RunnableLambda(  
    lambda x: x - 5  
)
```

```
chain = (  
    add_ten  
    | multiply_two  
    | subtract_five  
)
```

```
print(type(chain))
```

```
result = chain.invoke(5)
```

```
print(result)
```

14. How the Previous Program Executes

Initial input:

5

Step 1

add_ten

Calculation:

$5 + 10 = 15$

Output:

15

That output automatically becomes the input to:

multiply_two

Step 2

$$15 \times 2 = 30$$

Output:

30

That output becomes the input to:

subtract_five

Step 3

$$30 - 5 = 25$$

Final output:

25

Complete Flow

5

↓

add_ten

↓

15

↓

multiply_two

↓

30



subtract_five



25

This is **sequential execution**.

15. Inspecting RunnableSequence Steps

We can inspect different parts of the sequence.

```
print(chain.steps)
```

```
print(chain.first)
```

```
print(chain.last)
```

```
print(chain.middle)
```

Suppose:

a | b | c | d | e

Then:

first



a

middle

↓

[b, c, d]

last

↓

e

steps

↓

[a, b, c, d, e]

Simple Table

Property Meaning

first	First Runnable
middle	Runnables between first and last
last	Last Runnable
steps	Complete sequence of Runnables

16. Input/Output Compatibility

This is one of the **most important rules** in sequential workflows.

Output type of the previous step must be compatible with the input expected by the next step.

Example:

prompt | model | parser

Here the components are compatible.

Concept:

Previous Step Output



Must match



Next Step Input

17. Multi-Step AI Workflow

Suppose our requirement is:

Step 1

Generate an explanation.

Step 2

Based on that explanation, generate interview questions.

Architecture:

Topic



Generate Explanation



Explanation



Generate Questions



Questions

Now we have a true **multi-step workflow**.

18. First Prompt

```
prompt1 = PromptTemplate.from_template(
```

```
    """
```

```
    Explain {topic} in very simple English.
```

```
    Give a simple definition and
```

```
    5 important points.
```

```
    """
```

```
)
```

This prompt expects:

topic

Example input:

```
{"topic": "Python"}
```

19. Second Prompt

```
prompt2 = PromptTemplate.from_template(
```

```
    """
```

```
    Based on the explanation below,  
    create 5 beginner interview questions.
```

```
    Explanation:
```

```
    {explanation}
```

```
    """
```

```
)
```

This prompt expects a variable:

explanation

Therefore, its expected input is conceptually:

```
{
```

```
    "explanation": "..."
```

```
}
```

20. Compatibility Problem

Suppose we directly write:

prompt1 | model | parser | prompt2 | model | parser

After:

prompt1 | model | parser

the parser produces a string.

Example:

Python is a programming language...

But prompt2 contains:

{explanation}

So we prepare the correct dictionary structure.

21. Solving the Compatibility Problem with RunnableLambda

We use:

```
to_dict = RunnableLambda(  
    lambda explanation: {  
        "explanation": explanation  
    }  
)
```

Suppose the parser produces:

Python is a programming language.

RunnableLambda converts it into:

```
{  
  "explanation":  
    "Python is a programming language."  
}
```

Now this dictionary is suitable for:

prompt2

22. Correct Multi-Step Pipeline

prompt1

| model

| parser

| to_dict

| prompt2

| model

| parser

Complete Flow

Topic Dictionary



Prompt 1



Model



Parser



String Explanation



RunnableLambda



Explanation Dictionary



Prompt 2



Model



Parser



Final Questions

23. Multi-Step LLM Application

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
from langchain_core.runnables import RunnableLambda
```

```
model = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
parser = StrOutputParser()
```

```
prompt1 = PromptTemplate.from_template(  
    """"
```

Explain {topic} in very simple English.

Give a simple definition and
5 important points.

```
"""
```

```
)
```

```
to_dict = RunnableLambda(  
    lambda explanation: {  
        "explanation": explanation  
    }  
)
```

```
prompt2 = PromptTemplate.from_template(  
    """
```

```
Based on the explanation below,  
create 5 beginner interview questions.
```

Explanation:

```
{explanation}
```

```
"""
```

```
)
```

```
chain = (
```

```
    prompt1
```

```
    | model
```

```
    | parser
```

```
    | to_dict
```

```
    | prompt2
```

```
    | model
```

```
    | parser
```

```
)
```

```
result = chain.invoke(
```

```
{
```

```
        "topic": "Python"  
    }  
)
```

```
print(result)
```

24. Understanding the Complete Flow

Input:

```
{  
    "topic": "Python"  
}
```

Step 1:

Prompt 1

Step 2:

Model generates explanation

Step 3:

StrOutputParser converts model response to string

Step 4:

RunnableLambda

converts:

String

into:

```
{  
  "explanation": explanation  
}
```

Step 5:

Prompt 2

uses the explanation.

Step 6:

Model generates interview questions

Step 7:

Parser converts final model output to string

Final result:

Interview Questions

25. Can the Same Model Be Used More Than Once?

Yes.

Example:

chain = (

```
prompt1
| model
| parser
| to_dict
| prompt2
| model
| parser
)
```

The same:

model

is used two times.

26. Does One `chain.invoke()` Mean One LLM Call?

No.

This is very important.

We may write:

```
chain.invoke(...)
```

only once.

But inside the chain, the model may be called multiple times.

Example:

Prompt 1



Model Call 1



Prompt 2



Model Call 2

Therefore:

One chain.invoke()

does **not necessarily mean**:

One LLM call

27. Cost and Latency Awareness

Suppose we have:

Prompt 1 → Model

Prompt 2 → Model

Prompt 3 → Model

That may mean multiple model calls.

More model calls can mean:

More Tokens

+

More Cost

+

More Latency

Therefore:

Build multi-step workflows when the extra steps genuinely improve the application.

This is a very important production mindset.

28. Practical Application — Learning Material Generator

Requirement:

Step 1

Generate an explanation.

Step 2

Generate a quiz based on that explanation.

Flow:

Topic

↓

Generate Explanation



Explanation



Generate Quiz



Quiz

29. Learning Material Generator Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
from langchain_core.runnables import RunnableLambda
```

```
model = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

)

```
parser = StrOutputParser()
```

```
explanation_prompt = PromptTemplate.from_template(
```

```
    """
```

```
    Explain {topic} for a beginner.
```

```
    Use very simple English.
```

```
    Give one real-world example.
```

```
    """
```

```
)
```

```
prepare_quiz_input = RunnableLambda(
```

```
    lambda explanation: {
```

```
        "content": explanation
```

```
    }
```

)

```
quiz_prompt = PromptTemplate.from_template(
```

```
    """
```

```
    Based only on the following content,  
    generate 5 MCQs.
```

```
    Content:
```

```
    {content}
```

```
    """
```

```
)
```

```
chain = (
```

```
    explanation_prompt
```

```
    | model
```

```
    | parser
```

```
    | prepare_quiz_input
```

```
| quiz_prompt
| model
| parser
)

result = chain.invoke(
    {"topic": "Machine Learning"}
)

print(result)
```

30. Learning Material Generator Flow

```
{"topic": "Machine Learning"}
```

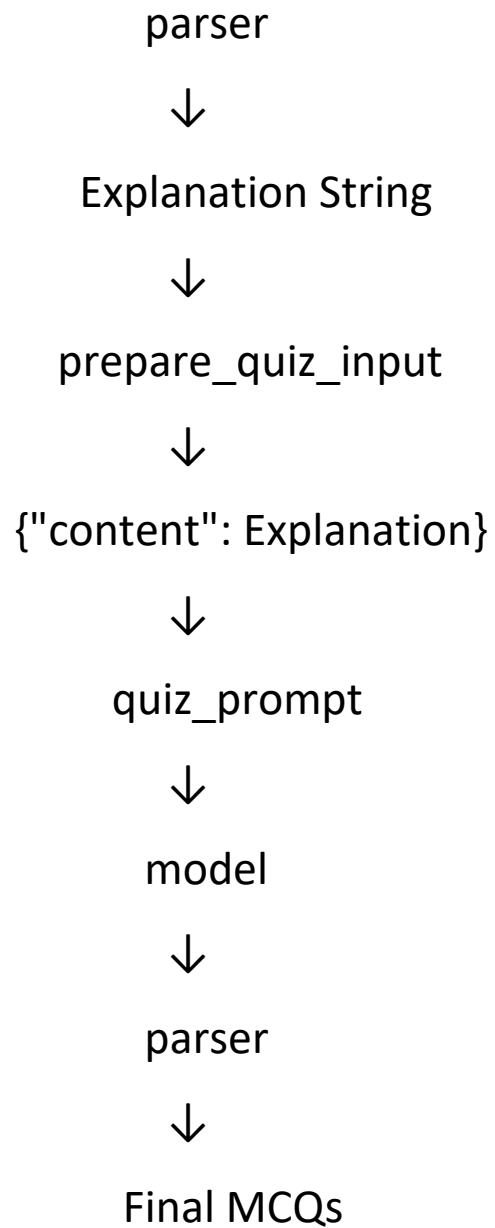


```
explanation_prompt
```



```
model
```





31. Practical Application — Blog Workflow

Requirement:

Topic



Generate Blog Outline



Use Outline



Generate Blog

Step 2 cannot properly proceed until Step 1 produces the outline.

Therefore, this is a perfect example of **sequential execution**.

32. Blog Generator Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
from langchain_core.runnables import RunnableLambda
```

```
model = ChatOpenAI(  
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
)
```

```
parser = StrOutputParser()
```

```
outline_prompt = PromptTemplate.from_template(
    """
    Create a short blog outline about {topic}.

    Give 5 headings.
    """
)
```

```
prepare_blog_input = RunnableLambda(
    lambda outline: {
        "outline": outline
    }
)
```

)

```
blog_prompt = PromptTemplate.from_template(
```

```
    """
```

```
    Write a beginner-friendly blog
```

```
    using this outline:
```

```
    {outline}
```

```
    Use simple English.
```

```
    """
```

```
)
```

```
chain = (
```

```
    outline_prompt
```

```
    | model
```

```
    | parser
```

```
    | prepare_blog_input
```

```
| blog_prompt  
| model  
| parser  
)
```

```
result = chain.invoke(  
    {  
        "topic": "Benefits of Artificial Intelligence"  
    }  
)
```

```
print(result)
```

33. Blog Workflow Execution

Topic



outline_prompt



Model



Parser



Outline String



RunnableLambda



{"outline": outline}



blog_prompt



Model



Parser



Final Blog

34. Day 3 Master Program

```
import os
```

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda
```

```
# -----
```

```
# Step 1: Create Model
```

```
# -----
```

```
model = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
```

```
parser = StrOutputParser()
```

```
# -----
```

```
# Step 2: Explanation Prompt
```

```
# -----
```

```
explanation_prompt = PromptTemplate.from_template(
```

```
    """
```

```
    Explain {topic} to a beginner.
```

```
    Requirements:
```

- Use very simple English
- Give a simple definition
- Give 5 important points
- Give one example

```
    """
```

```
)
```

```
# -----
```

```
# Step 3: Convert String to Dictionary
```

```
# -----
```

```
prepare_quiz_input = RunnableLambda(  
    lambda explanation: {  
        "explanation": explanation  
    }  
)
```

```
# -----
```

```
# Step 4: Quiz Prompt
```

```
# -----
```

```
quiz_prompt = PromptTemplate.from_template(  
    """
```

```
Based only on the explanation below,  
generate 5 MCQs.
```

```
For every MCQ provide:
```

- Question
- Four options

- Correct answer

Explanation:

{explanation}

"""

)

Step 5: Create Sequential Chain

chain = (

 explanation_prompt

 | model

 | parser

 | prepare_quiz_input

 | quiz_prompt

 | model

```
    | parser
)
```

```
# -----
```

```
# Step 6: Read Input
```

```
# -----
```

```
topic = input(
    "Enter topic: "
)
```

```
# -----
```

```
# Step 7: Execute Complete Sequence
```

```
# -----
```

```
result = chain.invoke(
    {
        "topic": topic
    }
)
```

```
}  
)
```

```
# -----
```

```
# Step 8: Display Result
```

```
# -----
```

```
print("\n-----")
```

```
print("FINAL QUIZ")
```

```
print("-----")
```

```
print(result)
```

35. Master Program Flow

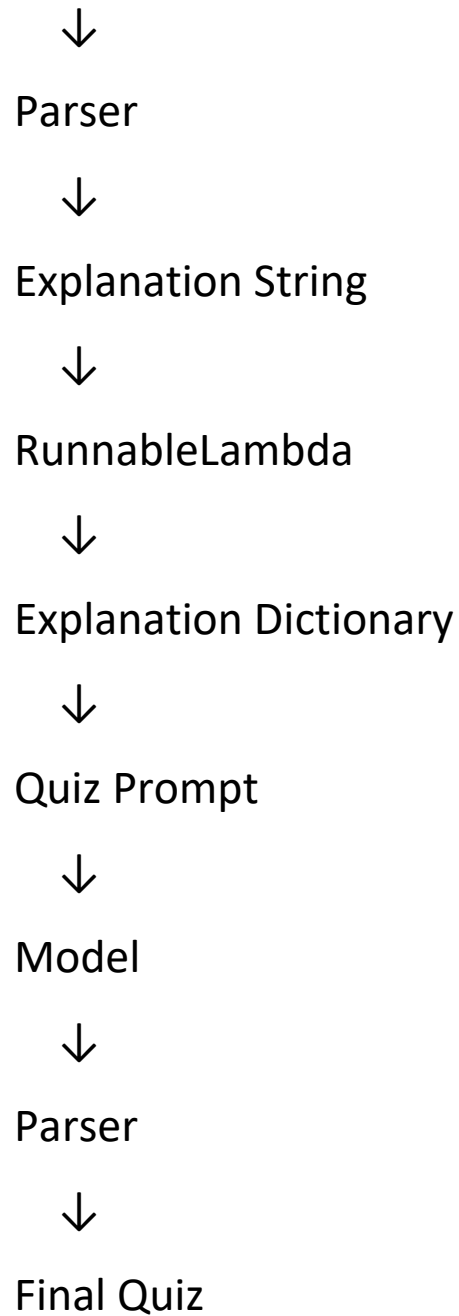
User Topic



Explanation Prompt



Model



36. batch() on RunnableSequence

We can also execute a complete RunnableSequence for multiple inputs.

Example:

```
inputs = [  
    {"topic": "Python"},  
    {"topic": "Java"},  
    {"topic": "SQL"}  
]
```

```
results = chain.batch(inputs)
```

Each input moves through the **entire sequence**.

37. How `batch()` Works with `RunnableSequence`

Python

Python



Whole Sequence



Result 1

Java

Java



Whole Sequence



Result 2

SQL

SQL



Whole Sequence



Result 3

Very important:

batch()

does not execute only one step.

Every input passes through the complete chain.

38. Sequence vs Independent Tasks

This is a very important decision.

Dependent Tasks

If there is a dependency between tasks:

A

↓

B

↓

C

use:

Sequential Workflow

or:

RunnableSequence

Example:

Generate Explanation

↓

Use Explanation

↓

Generate Quiz

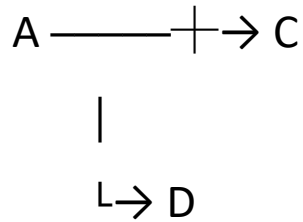
The second task depends on the first task.

39. Independent Tasks

If tasks are independent and execution order is not important:

$\vdash \rightarrow B$

|



use:

RunnableParallel

In independent tasks:

B does not depend on C

C does not depend on D

D does not depend on B

They can be handled independently.

40. RunnableSequence vs RunnableParallel

RunnableSequence

Tasks are dependent

One step follows another

Output of one can become input to next

Order is important

$A \rightarrow B \rightarrow C$

RunnableParallel

Tasks are independent

Multiple tasks can run independently

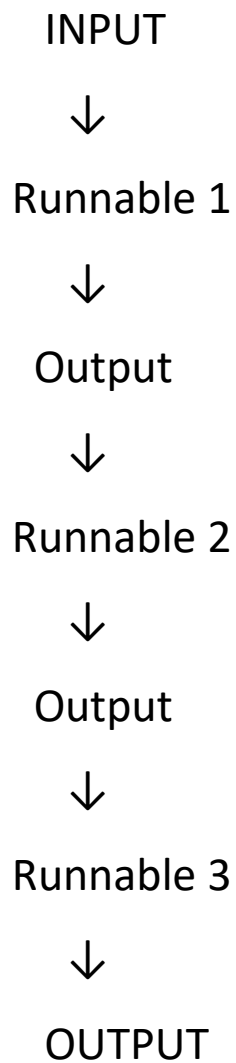
Tasks do not need previous task output

Order may not be important

$A \rightarrow B, C, D$

41. Master RunnableSequence Diagram

RunnableSequence



42. Master LCEL Diagram

prompt | model | parser

↓ ↓ ↓

Runnable Runnable Runnable

↓

RunnableSequence

43. Important Rule to Remember

Whenever we write:

a | b | c

think:

Input

↓

a

↓

Output of a

↓

Input to b



b



Output of b



Input to c



c



Final Output

44. Most Important Compatibility Rule

Before connecting two components:

A | B

ask:

Can the output produced by A be accepted as input by B?

If yes:

Compatible

If no:

Need Transformation

We can use something such as:

RunnableLambda

to transform the data.

45. Role of RunnableLambda in Our Multi-Step Workflow

Without transformation:

Parser



String



Prompt expecting Dictionary

Possible compatibility problem.

With RunnableLambda:

Parser



String



RunnableLambda



Dictionary



Prompt

Example:

```
lambda explanation: {  
    "explanation": explanation  
}
```

46. Final Revision Points

1. Sequential Execution

Operations happen:

One after another

in a defined order.

2. RunnableSequence

A RunnableSequence connects multiple Runnables sequentially.

Output of Previous



Input of Next

3. Pipe Operator

`a | b | c`

creates a sequential pipeline.

4. Explicit Constructor

We can also use:

```
RunnableSequence(  
    a,  
    b,  
    c  
)
```

5. RunnableLambda

RunnableLambda converts a Python callable into a Runnable.

Example:

```
RunnableLambda(  
    lambda x: x + 10  
)
```

6. Compatibility

The output of the previous Runnable must be compatible with the expected input of the next Runnable.

7. Multi-Step Workflow

Example:

Topic



Explanation



Interview Questions

8. Same Model Can Be Reused

A model can appear multiple times inside the same sequence.

9. One chain.invoke() Can Cause Multiple Model Calls

chain.invoke()



Model Call 1



Model Call 2

Therefore:

One invoke()

≠

Always one LLM call

10. More LLM Calls

More model calls can mean:

More Tokens

+

More Cost

+

More Latency

11. batch() with RunnableSequence

Every input passes through the complete sequence.

Input 1 → Whole Sequence → Result 1

Input 2 → Whole Sequence → Result 2

Input 3 → Whole Sequence → Result 3

12. RunnableSequence

Use when tasks are:

DEPENDENT

Example:

A

↓

B

↓

C

13. RunnableParallel

Use when tasks are:

INDEPENDENT

Example:

$\Gamma \rightarrow B$

$A \dashv \rightarrow C$

$L \rightarrow D$

47. Final Memory Formula

RUNNABLESEQUENCE

=====

Input



Runnable 1



Runnable 2



Runnable 3



Output

And remember:

Output of Left



Input of Right

One-Line Conclusion

RunnableSequence is used when multiple Runnables must execute one after another, where the output of one step becomes the input for the next step.

Very Simple Memory Line

DEPENDENCY



SEQUENCE



RunnableSequence

NO DEPENDENCY



INDEPENDENT TASKS



RunnableParallel